

2025 Fall CSED451 Assignment 4 Report

Team Name: openGameLab

Team Member #1: 안재영

컴퓨터공학과/20220019/enter21

Team Member #2: 정윤희

컴퓨터공학과/20240385/jyhyeok

1. Technical Details

개발 환경: Visual Studio 2022 버전 17.14.9 (July 2025)

cpp 확장자 파일을 powershell 스크립트 파일로 만든 후 powershell에서 빌드하여 구현 결과를 확인할 수 있다. 또한, OpenGL을 사용하는 코딩에 익숙해야 한다.

2. Implementation Details

2-1. Outline

이번 과제의 목표는 지금까지 구현한 Bullet Hell Shooter 게임의 전체 렌더링에 3D 셰이딩을 위한 GLSL 기반 렌더링 파이프라인을 구축하는 것이다. 기존에는 단순한 Opaque, Wireframe 스타일의 그래픽으로 오브젝트들을 표현하였으나, 이번 구현에서는 3D 조명 모델로 그래픽의 퀄리티를 개선한다. 새롭게 구현하는 그래픽 기능에는 3가지 종류의 셰이딩(Gouraud, Phong, Phong+Normal Mapping)과 Directional/Point Light, Normal Map 기반의 표면 디테일 표현과 텍스처 이미지 파일을 이용한 모델 렌더링 등이 포함된다.

이를 구현하기 위해 .vs, .fs 형식의 Shader 프로그램과 code 내에 확장된 Model 로딩 시스템, Tangent/Normal/UV 계산, 텍스처 로더 등을 새롭게 작성하였다. 특히 Model 클래스는 position 데이터만을 사용하던 기존의 수준에서 벗어나 pos, normal, uv, tangent를 포함한 11 float per vertex 구조의 VAO/VBO 기반 모델로 다시 작성하여 고도화된 렌더링에 필요한 정보들을 다루도록 하였다.

결과적으로 게임의 오브젝트들은 이전보다 복잡한 텍스처 및 광원 효과를 통해 공간감과 3D 입체감이 강화되고, 발전된 그래픽 결과물을 얻을 수 있었다.

2-2. Additional Requirements

- Multiple point light sources

fragment shader에서 PointLight 구조체를 담는 pointLights 배열을 선언한다. Assn4의 minimum goal과 달리 point light를 4개 사용하기 때문에, for문을 이용하여 각각의 point light 조명 계산을 한다. 이후 코드의 display 함수에서 조명의 위치, 색깔 등을 포함한 조명 4개의 정보들을 fragment shader로 전송한다. 또한 assn4의 다른 minimum goal인 gouraud shading의 경우 빛 계산이 vertex shader에서 관리되므로, vertex shader에도 fragment shader에서처럼 PointLight 구조체들을 담는 pointLights 배열 선언, for문을 이용한 빛 계산을 적용해주면 최종적으로 multiple point light sources를 구현할 수 있다.

- Planar shadow casting

배경이 검정색이므로 그림자는 회색으로 그린다. fragment shader에 isShadow bool 타입 변수를 추가하여, 그림자를 그릴 때와 아닐 때를 구분시킨다. 코드에서는 그림자를 그리는 전용 조명을 추가하여, $z = -1.0$ plane에 그림자가 평면으로 그려지도록 하였다. 그림자들을 그리기 위한 행렬은 generateShadow Matrix 함수에서 만들어진다. generateShadowMatrix 함수에서는 조명 위치와 바닥 평면으로 projection matrix를 생성하는 방식으로 행렬을 만들어낸다. 또한 기존 setGraphicStyleThenRender 함수의 앞부분에서 그림자를 그리도록 한다. 그림자의 경우 3D가 아니라 평면이기 때문에, 그림자를 그릴 때에 한해 깊이 버퍼를 끈다. 그림자를 그릴 때와 아닐 때를 isShadow bool타입 변수로 구분하여 이를 가능하게 하였다. 이렇게 그려지는 그림자는, 코드에서 조명 위치를 변경하여 크기와 위치를 적절히 조절할 수 있다.

- Spotlight

fragment shader에 SpotLight 구조체를 추가한다. 이 구조체는 기본적으로 PointLight가 가지고 있는 모든 변수들에 더하여 vec3 direction, float CutOff, float OuterCutOff를 추가적으로 가진다. direction은 스포트라이트를 비추는 방향, CutOff는 스포트라이트가 비추는 범위, OuterCutOff는 자연스러워 보이게 하기 위해 스포트라이트에서 비춰지는 빛이 흐려지는 범위를 의미한다. SpotLight 계산의 기본 골자는 PointLight 계산과 동일하나, theta, epsilon, intensity를 추가적으로 계산하여 스포트라이트를 비출 각도를 계산한다. 이 계산 과정은 fragment shader에 존재하는 calcSpotLight 함수에서 이루어진다. 그 후 코드에서 스포트라이트 정보를 셰이더로 넘겨준 후, 마지막으로 main함수에서 result에

스포트라이트 빛을 더해주면 스포트라이트 구현이 완료되고, 플레이어 비행기의 앞부분은 더 선명하게 보이게 된다.

2-3. Design Rationale

이번 변경의 핵심 목표는 기존 GLSL 기반 렌더링 파이프라인 위에 추가적인 3D 셰이딩 기법을 도입해 그래픽 퀄리티를 향상시키는 데 있다. 특히 Gouraud, Phong, Normal Mapping 세 가지 방식을 모두 구현하고, 이를 실시간으로 전환할 수 있도록 하여 조명 모델과 셰이딩 알고리즘의 차이점을 명확히 비교할 수 있도록 설계하였다.

이를 위해 먼저 각 셰이딩 방식마다 조명 계산이 이루어지는 위치(vertex 단계 or fragment 단계)가 다르다는 점을 고려해야 했다. 이 차이를 단순히 CPU 조건문으로 처리하는 방식은 유지보수가 어려우므로, 셰이더 프로그램 내부에 shadingMode라는 uniform 값을 전달하고, shader 내부 분기(if문)를 기반으로 공통 프로그램에서 세 가지 계산 방식을 모두 처리하는 형태로 구조를 잡았다. 이 방식은 모드를 전환하더라도 shader 재컴파일이 필요 없다는 장점도 있다.

또한 directional light와 point light의 조명 모델을 동시에 적용하기 위해 공통된 조명 계산 루틴을 설계해야 했다. directional light는 고정 방향 벡터만 필요하지만, point light는 위치 기반 계산과 attenuation 처리가 필요하므로 fragment shader 쪽의 조명 루틴이 자연스럽게 확장되었다. 특히 point light는 플레이어 주변을 공전해야 하므로, CPU에서 매 프레임 삼각함수로 위치를 갱신하고 glUniform3fv를 통해 셰이더에 전달하는 과정이 필요했다.

마지막으로 normal mapping 구현은 기존 Model/Node 기반 렌더링 구조와 조화를 이루도록 해야 했다. OBJ 로딩 단계에서 tangent 벡터를 계산할 수 있도록 Model::load 내부 로직을 확장하고, VAO/VBO 설정 단계에서 tangent attribute를 추가해 GPU로 전달할 수 있는 데이터 레이아웃을 마련하였다. 이를 통해 기존 drawRecursive 구조를 그대로 유지하면서 normal mapping 기능을 자연스럽게 추가할 수 있었다.

2-4. Design Implementation

(1) 셰이딩 모드 전환 시스템

handleKeyDown 함수에 G키 입력에 대한 처리를 추가하여 shadingMode 값을 $(\text{shadingMode} + 1) \% 3$ 방식으로 순환시킨다. shadingMode는 전역 변수로 선언되며 display() 실행 시 shader->setInt() 호출을 통해 그 shadingMode의 값을 GPU에 전달한다. drawRecursive 함수에서도 동일하게 shader->setVec3, shader->setMat4 등을 활용해 각 노드의 색상·변환 행렬을 셰이더로 업로드한다. 셰이더는 전달받은 shadingMode에 따라 vertex shader 또는 fragment shader에서 조명 계산을 선택적으로 수행한다.

(2) 광원 처리

셰이더 내부에 directional과 point 두 종류의 광원을 모두 처리할 수 있도록 공통 조명 루틴을 구성하였다. directional 광원의 경우 CPU 측에서 설정한 고정 방향 벡터와 색상을 uniform으로 전달받아 diffuse 및 specular 조연을 계산한다. 위치 정보가 필요 없으므로, fragment shader에서는 단순히 $\text{normalize}(-\text{dirLight.direction})$ 을 사용해 광원의 방향을 구한 후 Lambertian 및 Blinn-Phong 모델에 따라 조명 기여도를 산출한다.

반면 point 광원은 매 프레임 플레이어 오브젝트를 중심으로 공전하도록 요구되므로, 이를 위해 CPU에서 삼각함수를 사용해 플레이어 위치를 기반으로 한 조명 위치를 계산한 후 그 위치 pointLight.position과 색상, 그리고 attenuation에 필요한 상수 $(\text{pointLight.k0}, \text{k1}, \text{k2})$ 를 uniform으로 전달한다. fragment shader는 현재 fragment의 월드 좌표와 point 광원의 위치를 이용해 광원 방향 벡터를 계산하고, 거리 기반의 감쇠를 적용하여 최종 attenuation을 계산한다. 그 결과를 diffuse와 specular 항 모두에 곱해 최종 조명값을 결정한다.

결과적으로 fragment shader는 directional, point 각 조명의 diffuse 및 specular 값을 독립적으로 계산한 뒤, 기여도를 합산하여 픽셀에 대한 최종 조명 결과를 도출한다.

(3) normal mapping을 위한 tangent space 구성

OBJ 로딩 과정에서 Model::load 함수가 triangle 단위로 tangent를 계산하도록 수정한다. 두 개의 edge와 deltaUV를 사용해 tangent를 계산한 후에는 이를 vertex 구조체에 추가하고, glVertexAttribPointer(location=3)을 이용해 GPU로 전송한다.

또한 픽셀 단위의 조명 계산을 위해 fragment 셰이더에서 tangent와 Normal, 그리고 그 cross를 사용해 T, N, B를 생성하고 TBN 행렬을 구성한다. 그 계산 결과로 normal

map의 RGB 값을 tangent space normal로 변환한다.

(4) diffuse 및 normal map 텍스처 로딩

new_assets 폴더에 텍스처 파일들을 추가한 후 stb_image.h를 추가하여 stbi_load() 함수를 통해 로딩 기능을 구현하였다. png 형식의 텍스처 파일들을 로딩하며, 할당된 텍스처 ID는 Model의 textureID 또는 Node가 가진 textureID에 저장되며, drawRecursive에서 shader->setInt("diffuseMap", 0)로 바인딩된다. normal map은 동일한 방식으로 로딩하여 normalMap 텍스처 유닛에 연결한다.

(5) 셰이더의 구조 확장

vertex 셰이더는 model/view/proj 행렬을 조합한 gl_Position 계산 외에도 Gouraud 모드에 한한 per-vertex 조명 계산 후 vertexColor로 전달 기능, fragment shader의 TBN 구성을 위한 tangent 전달, normal mapping이 아닐 경우에 한한 normal 보간 기능을 구현하였다.

fragment 셰이더는 shadingMode 분기를 처리하며, normal mapping 여부를 판단해 TBN을 기반으로 normal을 결정한다. directional + point 조명의 diffuse/specular를 계산하며, diffuseMap 샘플링 적용하도록 구현한다. 결과적으로 전체적인 조명 계산의 대부분은 fragment 셰이더에서 수행되어 표면 표현의 퀄리티를 향상시킨다.

2-5. Additional Background

3D 셰이딩을 구현하기 위해서는 고전적인 텍스처 샘플링보다 더 높은 그래픽스 개념들에 대한 이해가 필요했다. 이번 구현에서 핵심적으로 다뤄진 배경 지식들은 다음과 같다.

(1) Gouraud, Phong 셰이딩 방식

두 기법 모두 고전적인 조명 모델에 기반하지만 계산의 위치가 다르다. Gouraud 셰이딩은 vertex 셰이더에서 조명을 계산하며, 보간된 색상이 fragment로 전달된다. 연산의 양은 상대적으로 적으나, 하이라이트 부분이 뚜렷하지 않다는 단점이 있다. phong 셰이딩은 fragment 셰이더에서 normal을 보간 후 조명을 계산하며, 연산량은 많지만 상대적으로 자연스럽게 부드러운 하이라이트를 구현할 수 있다. 실제로 게임 상에서 G키를 통해 셰이딩 모드를 전환하며 그 차이를 비교할 수 있다.

(2) Tangent space 조명

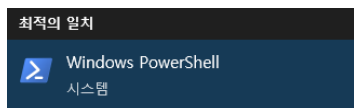
Normal mapping은 object space normal이 아닌 tangent space normal을 사용한다. 이를 위해 각 정점에 대해 tangent, bitangent, normal을 기반으로 한 TBN 행렬을 만들고, normal map의 RGB를 $[-1, 1]$ 범위의 벡터로 변환해 조명 계산에 적용한다. 참고로 이 과정은 Phong 셰이딩의 기반이므로 Gouraud 셰이딩에서는 사용되지 않는다.

(3) 조명 모델

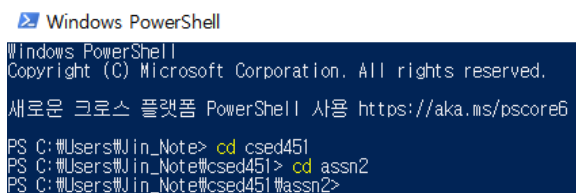
Directional, point 방식의 조명을 동시에 사용하기 위해 Lambertian diffuse, Blinn-Phong specular 모델을 함께 사용했다. 특히 point light는 거리에 따른 밝기 감쇠라는 특징을 가지고 있으므로, 셰이더 내부에서 distance 기반 감쇠 공식을 필요로 한다.

3. End-User Guide

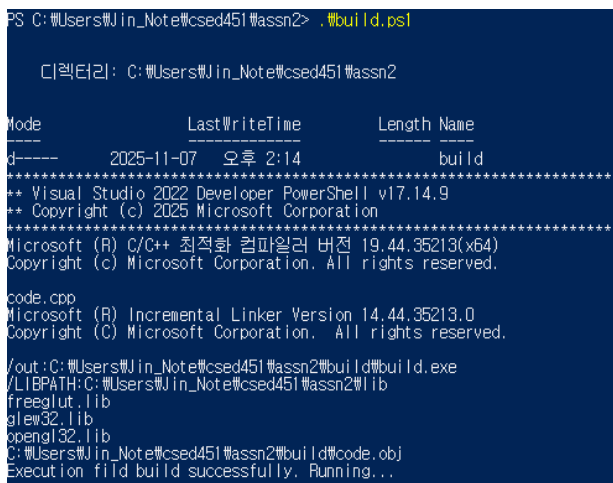
Windows powershell을 실행한다.



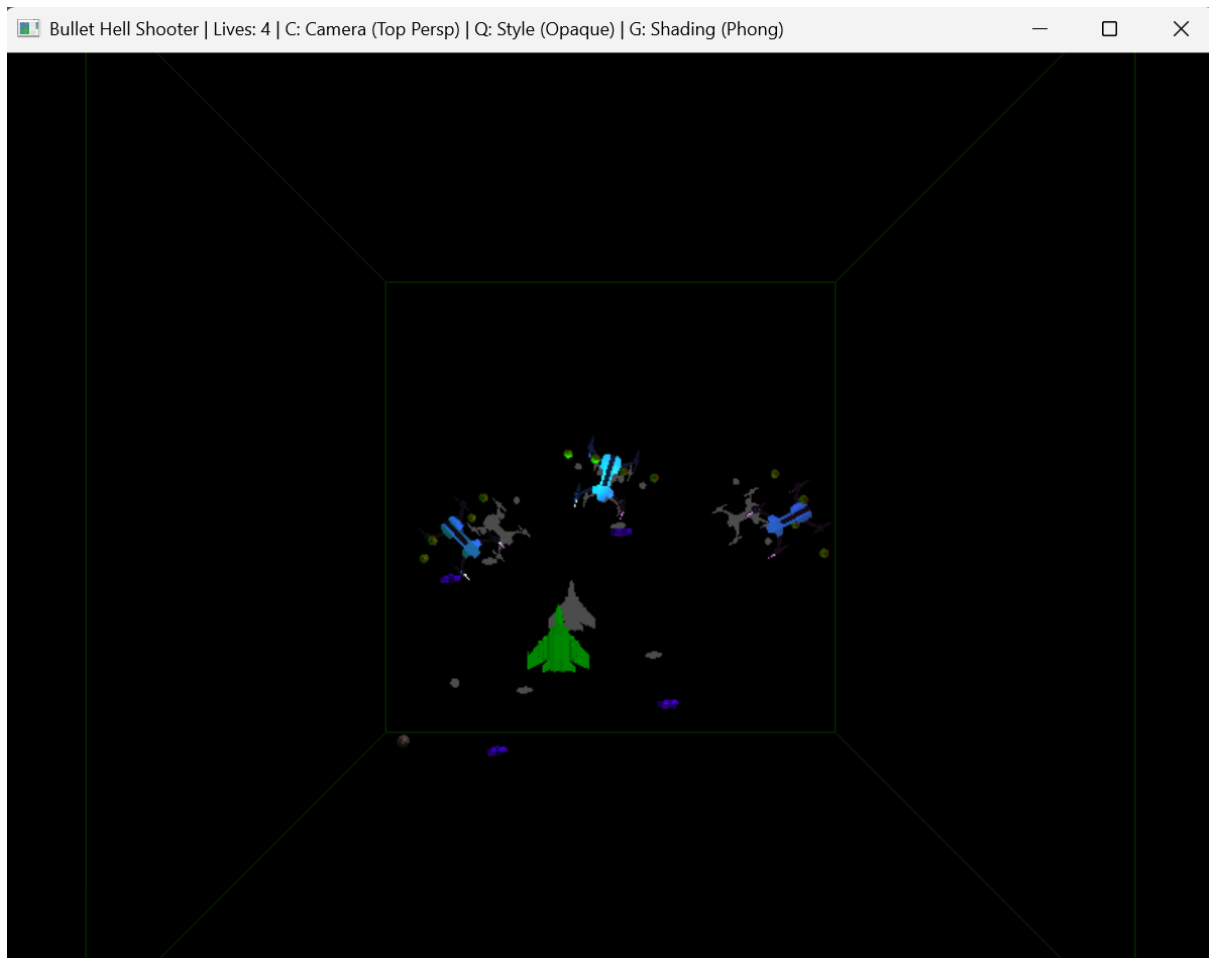
프로젝트가 저장된 파일 위치로 이동한다.



.\Wbuild.ps1을 입력한다.



잘 실행되는 것을 확인할 수 있다.



실행된 게임에서는 WASD 키를 통해 플레이어 이동이 가능하며, 스페이스바를 눌러 총알을 발사할 수 있다. Q키를 누르면 그래픽 스타일을, C키를 누르면 카메라 뷰 스타일을, 그리고 G 키를 누르면 셰이딩 스타일을 변경할 수 있다. 각 기능의 현재 상태는 윈도우 창 이름에 표시된다. 또한, 편리한 기능 확인을 위해 P키를 눌러 플레이어의 무적 상태를 On/Off로 전환할 수도 있다.

프로그램 종료 시 powershell 창에는 아래 메시지가 출력된다.

```
Execution exited.  
PS C:\Users\Jin_Note\csed451\wssn2>
```

이후 다시 빌드하여 프로그램을 실행시키는 것 또한 가능하다.

만약 Powershell 스크립트 실행 정책이 제한되어 있는 등의 이유로 Powershell에서 스크립트를 실행할 수 없다면, Get-ExecutionPolicy 명령어를 입력하여 실행 정책이 제한되어

있는지 확인해야 한다. 만약 그렇다면, Set-ExecutionPolicy RemoteSigned -Scope CurrentUser 명령어를 입력하여 스크립트 실행 정책을 변경해야 한다. 정책 여부 변경을 묻는 메시지가 나타나면 Y를 입력하여 스크립트 실행 정책을 변경하면 실행이 잘 된다.

```
PS C:\Users\Jin_Note> Set-ExecutionPolicy RemoteSigned -Scope CurrentUser
실행 규칙 변경
실행 정책은 신뢰하지 않는 스크립트로부터 사용자를 보호합니다. 실행 정책을 변경하면 about_Execution_Policies 도움말
항목 (https://go.microsoft.com/fwlink/?LinkID=135170)에 설명된 보안 위험에 노출될 수 있습니다. 실행 정책을
변경하시겠습니까?
[Y] 예(Y) [A] 모두 예(A) [N] 아니요(N) [L] 모두 아니요(L) [S] 일시 중단(S) [?] 도움말 (기본값은 "N"): Y
PS C:\Users\Jin_Note>
```

4. Discussions/Conclusions

4-1. Obstacles

additional goal 중 planar shadow casting을 구현할 때, 그냥 행렬만 만들었더니 오브젝트들의 그림자가 회전하는 point light의 빛을 받아 생겨서 바닥이 아닌 천장 쪽에서 회전하는 현상이 발생했었다. 또한, bounding box에도 그림자가 생기는 문제가 있었다. 이를 해결하기 위해 그림자를 그리기 전 bounding box를 잠깐 숨기고, point light 대신 planar shadow만을 위한 조명을 하나 새로 선언하였다.

4-2. Improvement Idea

현재 구현된 normal mapping은 tangent space 기반의 기본적인 방법을 사용하고 있으나, 보다 현실적인 표면 디테일을 위해 parallax mapping 또는 parallax occlusion mapping과 같은 고급 기법을 추가할 수 있다. 이러한 기법은 실제로 오브젝트 표면에 높낮이가 있는 것처럼 표현할 수 있어, 그 품질을 크게 향상시킬 수 있다.

또한 현재 게임 플레이 시에 생성되었다가 사라졌다 하는 추진 연료나 총알 등의 오브젝트 각각을 point 광원으로 설정한다면, 총알의 발사와 기체의 추진이 주는 박진감을 증진시키며 게임의 생동감을 높일 수 있을 것이다.

4-3. Lessons

기존 OpenGL 고정 파이프라인에서 GLSL 기반 프로그램 파이프라인으로 넘어오고, 그 후 새로운 기능들을 계속해서 추가하면서 CPU와 GPU가 서로 어떤 데이터를 어떻게 주고받는지에 대한 이해가 크게 깊어졌다. 단순히 glBegin/glEnd로 그리던 방식이 사라진 뒤, 모든 계산 흐름을 직접 구성해야 했던 경험이 실질적인 3D 그래픽스 파이프라인 이

해에 많은 도움이 되었다.

또한 다양한 셰이딩 기법을 구현해 비교하면서, 조명 계산이 어느 단계에서 수행되느냐에 따라 결과물이 얼마나 달라지는지, 그리고 tangent space가 왜 필요한지 명확하게 체감할 수 있었다. tangent 계산 과정에서 발생한 작은 오류 하나로 화면 전체에 예측하지 못한 오류가 발생하는 경험을 통해, 그래픽스 프로그래밍에서 수학적 정확성과 데이터 일관성의 중요성을 배울 수 있었다.

5. References

Learn OpenGL - <https://learnopengl.com/>

6. AI-assisted Coding References

이번 과제의 약 60%가 AI(Gemini 및 ChatGPT)의 도움을 받아 구현되었다. 셰이딩 로직 구조 설계, normal mapping 구현 방식, 오류 발생 부분 검증 등의 부분에서 구체적인 도움을 받았다.

7. individual contribution

이번 프로젝트는 두 팀원이 50:50 비율로 각자의 역할을 나누어 충실히 수행하였다. 각자 기본적인 3D 셰이딩의 구현과 추가적인 디테일로 파트를 나누어 프로젝트를 성공적으로 완료하였다.